

Computer Vision Lab Manual 02: Master Cheat-Sheet

Exhaustive Guide: Photometric / Point Transformations, Gamma Correction, Piecewise Linear Mapping & Histogram Processing

1. CORE THEORETICAL FOUNDATIONS & MATHEMATICAL FORMULATIONS

1.1 What are Photometric / Point Transformations?

Point Transformations (also known as intensity transformations or photometric mappings) represent the most fundamental spatial domain image processing operations. In a point transformation, the output intensity value **s** at a specific pixel coordinate (x, y) depends *strictly and solely* on the input intensity value **r** at that exact same coordinate. Unlike spatial filtering or convolution, point processing does **NOT** consider neighboring pixel values.

Mathematically, a point transformation is expressed as: **s = T(r)**, where *r* is the input intensity (e.g., 0 to 255 for an 8-bit image), *s* is the mapped output intensity, and *T* is the intensity transformation operator.

1.2 Mathematical Formulations of Intensity Transformations

Transformation Type	Mathematical Formula	Core Principle & Primary Use Case
Image Negative	$s = (L - 1) - r = 255 - r$	Inverts image intensity values. High intensities become low and vice versa. Ideal for enhancing white/light detail embedded in dark backgrounds (e.g., medical mammograms, chest X-rays).
Log Transformation	$s = c \cdot \ln(1 + r)$ where $c = 255 / \ln(1 + r_{max})$	Maps a narrow range of low-intensity dark input values into a wider range of output levels while compressing high-intensity bright values. Expands dark regions; crucial for displaying dynamic range compressed signals like Fourier Transform spectra .
Inverse Log Trans.	$s = c \cdot (e^r - 1)$	Opposite of log transformation; compresses dark values and expands bright intensity levels.
Power-Law (Gamma)	$s = c \cdot r^\gamma$ or $s = 255 \cdot (r / 255)^\gamma$	Adjusts image contrast dynamically based on exponent γ : • $\gamma < 1$ (Gamma < 1): Expands dark intensity values, brightens shadow regions. • $\gamma > 1$ (Gamma > 1): Compresses dark values, darkens image, highlights bright spots. Used for CRT monitor calibration (gamma ~ 2.2) and display device correction.
Contrast Stretching	Piecewise linear mapping: $s = [(s2 - s1)/(r2 - r1)] \cdot (r - r1) + s1$	Linearly stretches pixel intensity values spanning a narrow interval [r1, r2] to cover the full dynamic range [0, 255]. Dramatically improves contrast in poorly illuminated scenes.
Intensity-Level Slicing	Type A (Binary): $s = L-1$ if $A \leq r \leq B$ else 0 Type B (Preserve): $s = L-1$ if $A \leq r \leq B$ else r	Highlights a specific range of intensity levels [A, B] of interest (e.g., isolating blood vessels or tumor boundaries) while either suppressing (Type A) or preserving (Type B) background pixels.
Bit-Plane Slicing	$b_k = (r \gg k) \& 1$ or $b_k = \text{bitwise_and}(r, 2^k)$	Decomposes an 8-bit image into 8 distinct binary planes (Bit 0 LSB to Bit 7 MSB). Higher-order bits (Bit 7, 6, 5) contain most visual structural energy; lower bits contain noise.
Histogram Equalization	$s_k = T(r_k) = (L - 1) \cdot \sum_{j=0}^k \{p_r(r_j)\}$ $= \text{round}(255 \cdot \text{CDF}(k))$	Transforms the intensity distribution of an image so that its resulting histogram is flat/uniform, maximizing global image contrast by spreading out frequent intensity values.
CLAHE (Adaptive Eq.)	Local AHE with Clip Limit contrast capping	Divides image into M x N contextual tiles (e.g., 8x8), equalizes each tile independently, and clips histogram height above a threshold to prevent noise over-amplification in uniform regions.

1.3 Real-World Applications Matrix

Application Domain	Target Task	Key Objective & Technical Challenge
Medical Diagnostic Imaging	Mammograms, Chest X-Rays, Bone Scans	Applies Image Negative and Contrast Stretching to reveal subtle tissue micro-calcifications and bone hairline fractures embedded in dark backgrounds.
Frequency Domain Processing	Fourier Transform Magnitude Display	Fourier spectra have extreme dynamic range (e.g., 0 to 10^6). Log transformation compresses the huge dynamic range into readable [0, 255] display levels.
Display Device Calibration	CRT, OLED, & Scanner Gamma Correction	Displays exhibit non-linear intensity response ($s = r^{2.2}$). Gamma correction ($\gamma = 1/2.2 \sim 0.45$) pre-distorts image to achieve true linear visual output.

Application Domain	Target Task	Key Objective & Technical Challenge
Satellite & Aerial Imagery	Terrain, Forest & Water Body Segmentation	Uses Intensity-Level Slicing and CLAHE to highlight specific water/vegetation spectral bands without over-amplifying atmospheric haze noise.
Data Compression & Steganography	Bit-Plane Decomposition & Watermarking	Bit-plane slicing proves Bit 7 and 6 contain >85% of image structural data, allowing compression by discarding Bit 0-2 or hiding watermarks in LSB.

2. COMPREHENSIVE TERMINOLOGY DICTIONARY (22 KEY TERMS)

Term & Definition (1-11)	Term & Definition (12-22)
1. Intensity / Point Transformation: A spatial domain image operation mapping an input pixel intensity r to an output intensity s via $s = T(r)$, independent of adjacent neighboring pixels.	2. Monotonic Transformation Function: A transformation function $T(r)$ that is strictly increasing (or non-decreasing), ensuring that the relative brightness order of gray levels is preserved and preventing intensity inversion artifacts.
3. Dynamic Range: The ratio or range between the maximum and minimum measurable pixel intensity values in an image system (e.g., 0 to 255 for 8-bit unsigned integer images).	4. Image Negative: An intensity transformation function $s = (L - 1) - r$ that inverts grayscale levels, making dark regions light and light regions dark.
5. Logarithmic Transformation: A non-linear transformation $s = c \cdot \ln(1 + r)$ that expands dark pixel values while compressing bright values, widely used for Fourier spectrum display.	6. Gamma / Power-Law Correction: A transformation $s = c \cdot r^\gamma$ used to correct non-linear response curves of display devices or adjust image brightness/contrast dynamically.
7. Look-Up Table (LUT): A 1D array of pre-computed mapped intensity values (length 256) allowing $O(1)$ constant-time point transformation execution via <code>cv2.LUT()</code> .	8. Piecewise Linear Transformation: A transformation function composed of connected linear segments, allowing customized, non-uniform mapping across different intensity sub-intervals.
9. Contrast Stretching: A piecewise linear operation that expands the dynamic range of an image by stretching a narrow band of input intensities $[r_1, r_2]$ to span $[0, 255]$.	10. Intensity-Level Slicing: A technique that highlights a specific band of intensity values $[A, B]$ while either setting all other pixels to 0 (binary) or retaining them (background-preserving).
11. Bit-Plane Slicing: Decomposing an 8-bit image into 8 individual binary images, where each plane represents the contribution of a specific bit position (Bit 0 LSB to Bit 7 MSB).	12. Image Histogram: A discrete statistical graph plotting the frequency count n_k of pixels occurring at each intensity level r_k across an image plane.
13. Probability Density Function (PDF): In digital images, the normalized histogram $p(r_k) = n_k / (M \cdot N)$, representing the probability of occurrence of intensity level r_k .	14. Cumulative Distribution Function (CDF): The running sum of normalized histogram probabilities up to level k : $CDF(k) = \sum_{j=0}^k p(r_j)$, used directly as the histogram equalization mapping function.
15. Histogram Equalization (HE): A global image enhancement method that redistributes pixel intensities based on the image CDF to achieve a uniform/flat output histogram and maximize global contrast.	16. Histogram Matching (Specification): A technique that transforms an image's intensity distribution to match a specified target histogram shape rather than a flat uniform distribution.
17. Adaptive Histogram Equalization (AHE): A local contrast enhancement technique that computes histogram equalization independently over small contextual image sub-regions (tiles).	18. CLAHE (Contrast Limited AHE): An advanced variant of AHE that limits local contrast enhancement by clipping histogram bins above a specified threshold (<code>clipLimit</code>) to avoid over-amplifying noise.
19. Clip Limit: A parameter in CLAHE defining the maximum allowable height of histogram bins in local tiles. Excess pixels above the clip limit are redistributed uniformly across all bins.	20. Tile Grid Size: The spatial dimensions (e.g., 8x8) of local contextual regions into which an image is partitioned for local adaptive histogram equalization in CLAHE.
21. Brightness Preserving Equalization: Advanced histogram equalization variants (e.g., BBHE, DSIHE) that split the histogram around the mean or median intensity before equalizing to preserve original overall brightness.	22. Dynamic Range Compression: Compressing a vast numerical range of data (e.g., 0 to 10^6 in physical light sensors or Fourier domains) into a displayable 8-bit scale $[0, 255]$.

3. EXHAUSTIVE FUNCTION & PARAMETER BREAKDOWN REFERENCE

Function 1: cv2.LUT(src, lut, dst=None)

Performs fast constant-time $O(1)$ intensity transformation mapping on an array using a pre-calculated 1D Look-Up Table array.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (<code>uint8</code>)	Input image array (single-channel or multi-channel 8-bit unsigned integer matrix).
lut	ndarray (<code>uint8</code> , shape 256x1)	1D Look-Up Table array containing 256 mapped values where index i stores mapped intensity $T(i)$.
dst	ndarray (optional)	Output array of same size and depth as <code>src</code> .

Function 2: cv2.calcHist(images, channels, mask, histSize, ranges)

Calculates the statistical intensity histogram frequency array for a set of images.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
images	list of ndarrays [<code>uint8</code>]	Source images provided as a Python list e.g., <code>[image]</code> .

Parameter	Data Type / Range	Detailed Description & Mathematical Role
channels	list of ints e.g. [0]	Index of channel to compute histogram for: [0] for grayscale, [0, 1, 2] for BGR.
mask	ndarray or None	Optional 8-bit mask image. If provided, histogram is computed only for non-zero mask pixels.
histSize	list of ints e.g. [256]	Number of histogram bins for each channel e.g., [256] for full 8-bit intensity resolution.
ranges	list of floats e.g. [0, 256]	Intensity measurement range bounds [0, 256] (upper bound is exclusive: 0 to 255).

Function 3: cv2.equalizeHist(src, dst=None)

Equalizes the histogram of a grayscale image using global cumulative distribution function (CDF) mapping.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray (‘uint8’)	Input 8-bit single-channel grayscale image array.
dst	ndarray (optional)	Output equalized grayscale image array of exact same shape and depth as src.

Function 4: cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

Instantiates a Contrast Limited Adaptive Histogram Equalization (CLAHE) object.

Parameter	Data Type / Range	Detailed Description & Mathematical Role
clipLimit	float (default = 2.0)	Threshold for contrast clipping in local histogram tiles. Higher values increase contrast but risk noise amplification.
tileGridSize	tuple of ints (e.g., (8,8))	Grid size for splitting image into contextual sub-regions e.g., (8,8) creates 64 independent tiles.

Function 5: cv2.normalize(src, dst, alpha, beta, norm_type)

Normalizes array intensity values to span a specified output scalar range (used for Contrast Stretching).

Parameter	Data Type / Range	Detailed Description & Mathematical Role
src	ndarray	Input image array to normalize.
dst	ndarray	Output normalized array.
alpha	float (e.g. 0)	Lower range boundary (or norm value) for normalization.
beta	float (e.g. 255)	Upper range boundary for normalization when using cv2.NORM_MINMAX.
norm_type	OpenCV Norm Flag	Normalization type: cv2.NORM_MINMAX scales values linearly to range [alpha, beta].

4. FULLY ANNOTATED CODE IMPLEMENTATIONS FOR LAB 02

Program 1: Basic Intensity Transformations (Image Negative, Log Transformation & Fast Gamma Correction via LUT)

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load grayscale image
gray_img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# A. IMAGE NEGATIVE TRANSFORMATION:  $s = (L - 1) - r = 255 - r$ 
negative_img = 255 - gray_img

# B. LOGARITHMIC TRANSFORMATION:  $s = c * \log(1 + r)$ 
# Calculate scaling constant c so max output equals 255
c_log = 255.0 / np.log(1.0 + np.max(gray_img))
# Apply logarithmic mapping (float conversion prevents uint8 overflow during log)
log_img = c_log * np.log(1.0 + gray_img.astype(np.float64))
log_img = np.uint8(np.clip(log_img, 0, 255))

# C. POWER-LAW / GAMMA CORRECTION USING FAST LOOK-UP TABLE (LUT)
def apply_gamma_correction(image, gamma=0.5):
    # Construct 256-element Look-Up Table (LUT)
    # Formula:  $s = 255 * (r / 255) ^ \text{gamma}$ 
    inv_gamma = gamma
    lut = np.array([((i / 255.0) ** inv_gamma) * 255 for i in range(256)]).astype(np.uint8)
    # Apply LUT mapping in O(1) time per pixel using cv2.LUT
    return cv2.LUT(image, lut)

# Gamma < 1: Expands dark values (brightens shadow regions)
gamma_bright = apply_gamma_correction(gray_img, gamma=0.4)
# Gamma > 1: Compresses dark values (darkens image, highlights bright spots)
gamma_dark = apply_gamma_correction(gray_img, gamma=2.2)

```

Program 2: Piecewise Linear Transformations (Contrast Stretching, Intensity Slicing & Bit-Plane Slicing)

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

# A. CONTRAST STRETCHING (Min-Max Normalization to [0, 255])
# Automatically maps min(I) -> 0 and max(I) -> 255
contrast_stretched = cv2.normalize(image, None, alpha=0, beta=255, norm_type=cv2.NORM_MINMAX)

# B. INTENSITY-LEVEL SLICING
# Highlight intensity range [A, B] = [100, 180]
A, B = 100, 180
# Type A (Binary Slicing): Target range set to 255, all other pixels set to 0
slicing_binary = np.zeros_like(image)
slicing_binary[(image >= A) & (image <= B)] = 255

# Type B (Background-Preserving Slicing): Target range set to 255, others retained
slicing_preserve = image.copy()
slicing_preserve[(image >= A) & (image <= B)] = 255

# C. 8-BIT PLANE SLICING
bit_planes = []
for k in range(8):
    # Extract bit k using bitwise AND with  $2^k$ , then scale bit to 255 for display
    plane = cv2.bitwise_and(image, 2**k)
    plane_binary = np.uint8((plane > 0) * 255)
    bit_planes.append(plane_binary)
# bit_planes[7] is MSB (Bit 7 - structural content), bit_planes[0] is LSB (Bit 0 - noise)

```

Program 3: Global & Adaptive Histogram Equalization (cv2.equalizeHist & CLAHE)

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

gray_img = cv2.imread('low_contrast.jpg', cv2.IMREAD_GRAYSCALE)

# 1. CALCULATE HISTOGRAM AND CDF MANUALLY
hist = cv2.calcHist([gray_img], [0], None, [256], [0, 256])
pdf = hist.ravel() / hist.sum()
cdf = pdf.cumsum()

# 2. GLOBAL HISTOGRAM EQUALIZATION
# Applies global CDF mapping across entire image plane
global_eq = cv2.equalizeHist(gray_img)

# 3. CONTRAST LIMITED ADAPTIVE HISTOGRAM EQUALIZATION (CLAHE)
# Prevents noise over-amplification by equalizing local 8x8 contextual tiles
clahe_obj = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
clahe_eq = clahe_obj.apply(gray_img)

# Display comparisons
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(gray_img, cmap='gray'); axes[0].set_title('Original Image')
axes[1].imshow(global_eq, cmap='gray'); axes[1].set_title('Global Histogram Eq.')
axes[2].imshow(clahe_eq, cmap='gray'); axes[2].set_title('CLAHE Adaptive Eq.')
plt.tight_layout(); plt.show()

```

5. QUICK REFERENCE & EXAM SURVIVAL KIT

5.1 Master Comparison Matrix of Intensity Transformations

Method / Technique	Input Domain	Key Strengths	Primary Weakness / Limitation
Image Negative	Grayscale Image	Inverts contrast; reveals light features in dark areas.	Destroys dark background context completely.
Log Transformation	Grayscale / Fourier Spectrum	Expands dark values; compresses huge dynamic ranges.	Requires floating point math & scaling constant c.
Gamma Correction	Grayscale / Color Image	Dynamic non-linear brightness adjustment ($\gamma < 1$ or $\gamma > 1$).	Requires tuning gamma exponent parameter.
Contrast Stretching	Low Contrast Image	Spans narrow intensity range across full [0, 255].	Sensitive to extreme outlier noise pixels (min/max).
Intensity Slicing	Grayscale Image	Isolates specific intensity bands of interest.	May eliminate ambient background context (Type A).
Bit-Plane Slicing	8-Bit Grayscale Image	Isolates bit contributions; ideal for data compression.	Lower bits (0-2) appear as random noise.
Global Histogram Eq.	Low Contrast Grayscale	Parameter-free global contrast maximization.	Over-amplifies background noise; causes unnatural washed-out look.
CLAHE Adaptive Eq.	Grayscale Image	Superior local contrast enhancement; caps noise via clipLimit.	Requires tuning clipLimit and tileGridSize parameters.

5.2 High-Yield MCQ Traps & Exam Trick Alerts

- **Trap 1 (Log Transform Floating Point Overflow):** Applying `np.log(image)` directly on uint8 array causes runtime crash or divide-by-zero if pixels are 0. Always calculate `np.log(1.0 + image.astype(np.float64))`!
- **Trap 2 (Gamma Power-Law Float Range):** Formula $r^{**} \text{ gamma}$ fails on uint8 integers. You MUST normalize input pixels to float [0.0, 1.0] by dividing by 255.0 before exponentiation: `((img / 255.0) ** gamma) * 255!`
- **Trap 3 (Fast Execution with LUT):** Looping over image pixels using nested Python for loops takes seconds! Always use `cv2.LUT(image, lut)` for O(1) execution speed.
- **Trap 4 (Global Histogram Eq. Noise Artifacts):** Global `cv2.equalizeHist()` on homogeneous images with small noise spikes over-amplifies noise dramatically. Use `cv2.createCLAHE(clipLimit=2.0)` instead.
- **Trap 5 (Color Histogram Equalization Mistake):** Equalizing BGR channels independently alters image color hues dramatically! Correct approach: convert BGR to HSV or YCrCb, equalize ONLY the Value (V) or Luminance (Y) channel, and convert back.

5.3 'Which Function Do I Use?' Decision Matrix

Exam Problem Requirement	Primary Function / API	Key Parameters & Flags
Display Fourier spectrum / wide dynamic range	<code>c * np.log(1.0 + img)</code>	<code>c = 255.0 / np.log(1.0 + np.max(img))</code>

Exam Problem Requirement	Primary Function / API	Key Parameters & Flags
Correct CRT display / adjust dark shadows	cv2.LUT(img, lut)	lut = np.uint8(((np.arange(256)/255.0)**gamma)*255)
Stretch low contrast image to [0, 255]	cv2.normalize()	alpha=0, beta=255, norm_type=cv2.NORM_MINMAX
Highlight intensity band [100, 180]	Numpy boolean indexing	mask[(img >= 100) & (img <= 180)] = 255
Extract Bit 7 MSB structural plane	cv2.bitwise_and()	cv2.bitwise_and(img, 128) or (img >> 7) & 1
Global contrast equalization	cv2.equalizeHist()	src (8-bit single-channel grayscale image)
Local contrast enhancement without noise	cv2.createCLAHE()	clipLimit=2.0, tileGridSize=(8,8)

5.4 Common Coding Errors & Syntax Fixes

Observed Error / Runtime Bug	Root Cause Analysis	Exam Fix / Solution Syntax
TypeError: src is not a numpy array	Passing list or non-array object to OpenCV function.	Wrap array explicitly: np.array(image, dtype=np.uint8).
cv2.error: depth == CV_8U in equalizeHist	Passing 3-channel color image or float array to equalizeHist.	Convert to single-channel grayscale uint8 first: cv2.cvtColor(img, cv2.COLOR_BGR2GRAY).
Distorted colors after equalization	Applying equalizeHist on BGR channels independently.	Convert to HSV, equalize V channel: hsv[:, :, 2] = cv2.equalizeHist(hsv[:, :, 2]).
Log transform output is completely white	Forgot to normalize or multiply by scaling constant c.	Scale log values: c = 255.0 / np.log(1.0 + np.max(img)).